An Exploration of the Development and Current Applications of AI in Automated Theorem Proving

Alessandro Felici, 19 April 2025

Abstract

Artificial intelligence (AI) models have very rapidly become powerful and widely accessible tools. As a result, mathematicians have become interested in discovering how this tool can be used to create productivity and innovation in the field of automated theorem proving (ATP). While met with challenges such as the black box problem, lacking in reference data, and varying error prone rates, there are possibilities to make use of the current state of AI tools, and growing potential as it evolves to improve on its weaknesses. In this paper, I have researched current arguments and experiments in the potential of AI tools benefiting mathematical proving. I discuss the history of ATP leading up to research with implementing AI into autonomous automated theorem proving (AATP), and how it may be used to increase efficiency of proving in its current state. Then, I focus on a comparison between two strategies which incorporate AI tools in proving a set of problems to demonstrate the effectiveness of different methods.

Introduction

Automated theorem proving (ATP) is a field in mathematics in which machines help mathematicians with proofs by deducing logic using an algorithmic procedure. This is a subfield of automated reasoning, but it is common to focus on reasoning through mathematical proving. The goal of this project is to explore ATP and gain deeper knowledge into this specific area of discrete mathematics, then to research the possibilities offered by artificial intelligence (AI). As a result, I will draw unique conclusions using the sources from my investigation. The research in this field expands on the uses of propositional logic and equations used in AI models, but also requires further exploration into topics such as Gödel's incompleteness theorem, Monte Carlo tree search, reinforcement learning, and artificial neural networks (ANN). Then, it is also a goal of this project to explain these topics within the context of ATP to a general audience with a background in discrete mathematics related to computer science.

Given the current state of AI models, it is of interest to discover how it can be used to assist mathematicians, beyond the mundane ways it is often used. Developing the use of AI in this area provides an opportunity to observe how different models can be used in a cohesive system, where it can be built into current procedures, and how its implications in mathematics may reflect its development in a wider scope. The challenges highlighted in this area offer insight on implementing AI models into other fields of work. To start, I will give a brief overview of the recent history in ATP. This description begins with a background of the mathematical tools which came before the development of this field, along with the motivation to continue research on machines in mathematical proving. Then, I will discuss the use of current AI models in ATP, considering the strengths and weaknesses of the present state of these tools. It will focus on

machine learning and ANN in particular, with consideration to current rule-based proof assistants. The discussion of these models will include a review of experimental results in the efficiency of using machine learning in ATP. Provided this background, I will discuss an example which demonstrates the strengths and weaknesses of using two different methods to prove a set of conjectures.

Background

ATP is commonly used in two different ways: interactive theorem proving (ITP) and autonomous automated theorem proving (AATP). In the case of ITP, the mathematician uses automated reasoning to verify a given theorem. These are also referred to as proof assistants. This approach involves human-machine collaboration in order to develop confidence in the result. On the other hand, AATP takes only axioms as input, and may provide a new proof and theorem without human involvement in the process. ITP and AATP may be used with the same software by the same mathematician, as only the implementation separates these two methods. In opposition to the potential of ATP are Gödel's incompleteness theorems. The first theorem which Gödel provides claimed that, in a formal system, there will always be some statement which can never be proved or disproved within the system [1]. As mathematicians considered developing proof-assistants, this theorem stood as a clear argument against the capabilities of machines in proving.

The Monte Carlo tree search and reinforcement learning are two relevant topics under the field of AI, specifically in machine learning. These ideas appear later in the paper with reference to the results of an experiment on the efficiency of reinforcement learning in theorem proving. The Monte Carlo tree search is an algorithm for making decisions which uses random sampling and calculations within a reduced scope. It is frequently used to simulate AI in playing board games. The process follows four steps: calculate the highest value move given a set of possible nodes to choose from, move to the chosen node, simulate the rest of the game with random choices, and record the value of this path based on the simulations [2]. Reinforcement learning represents an algorithm which allows a machine learning system to dynamically communicate with its environment. In this situation, the environment contains variables, boundary values, rules, and possible actions. Then, the system will receive reward information from its environment, and make a decision based on this [3].

History and Development of ATP

Although the modern description of computers is frequently associated with high-level abstractions, mathematicians have developed many simple computing devices long before this point. Devices such as the abacus, trigonometry tables, and logarithmic tables were originally used as a method of simplifying the search for a solution. In the past century, mathematicians sought to develop this method, so that mathematical proofs could be automated. This would require the formalization of mathematics. However, while working towards this goal, Kurt Gödel published his work on the undecidability of formal systems, as described in his two incompleteness theorems. Then, it would not be possible to formalize math, and consequently it may not be possible to fully utilize automated reasoning. Despite this, mathematicians continued

to develop the field of automated reasoning, encountering early successes beginning in the 1950's.

Rather than search for a solution, computers could be used to directly improve on the work of mathematicians. Computers would be formed around propositions and axioms, thus given the ability to think non-numerically. The phrase AI could easily be connected to these machines, as it could be debated that this initial development allows the system to demonstrate properties of the human mind. Theorem proving software was developed as a form of these tools, some common programming languages being lean, Mizar, and E. Even with the initial debate on intelligence, these systems remained to be rule-following procedures, and could not be separated from machine level intelligence. Given that the code was written properly, these rule-based provers provide clear, comprehensible procedures and trustworthiness in their output. Its mechanics are also limiting however, as this process makes it inherently difficult for AATPs to distinguish interesting and trivial proofs, defined as a central challenge in ATP in a journal by Markus Pantsar [4]. Mathematicians may term a proof to be beautiful or creative, in which theorem provers lack the ability to think like a human in order to come to these conclusions. It would not come until further development of machine learning, such as in ANN, that researchers would make a distinction between AI demonstrating human-like behavior and thinking in a human-like fashion.

Artificial Neural Networks and Machine Learning

The current generation of rule-based proof assistants is constrained by what is programmable. However, AI tools such as machine learning with ANN provide a different approach. Stated previously, one central issue to rule-based theorem provers is their inability to classify a proof as interesting or trivial. There have been some efforts to accomplish this, starting by analyzing the length of a proof output by an AATP. Even with this, it is nevertheless difficult to create code which can describe a proof as creative. The unique construction of ANN, replicating the structure of human neural networks, shows the potential of thinking like a human. Given this setup, using this tool in AATP may work around the limitations in rule-based proof assistants. Also, in ITP, it can assist in gathering information, such as finding relevant statements for proving a conjecture. The focus in this section will be on its uses in AATPs. In addition to potentially recognizing interesting proofs, it can point out details which may be overlooked.

For example, machine learning was used to make connections in knot theory, discussed in a presidential lecture by Terence Tao [5]. Without this tool, mathematicians would give knots a number referred to as invariance, which describes the empty space outside of the knot, often implemented using a signature, based on a combinatorial set of equations. The signature is made up of twenty integers which each represent a unique aspect of the knot, so the signature could be varied and run through the model to experiment how the output would differ with different inputs. Trained on invariance, the model was able to accurately find the signature of the knot. More importantly, this method highlighted something mathematicians did not expect—the factors they were not expecting to be influential had a large impact on the invariance. This

illustrates the ability of ANN to accomplish learned mathematics, and how it may be possible to develop the ability to think in a human-like fashion. This model creates opportunities to explore which were not available with previous generations of theorem provers.

Additionally, ANNs are capable of completing more proofs when used as an AATPs in comparison to rule-based AATPs. A group of members from different universities experimentally showed an increase in efficiency by creating a control group with theorem prover leanCoP, then implementing machine learning through Monte Carlo guided tree searching, policy learning, and value learning [6]. In this experiment, the prover is input with axioms and a conjecture. It produces a set of clauses which are atomic propositions, and begins the process with a start clause. It progresses with extension or reduction steps, building a binary tree with the given clauses. The branch it is searching through may be extended if there are more literals in the clause, or closed otherwise, respectively for the extension or reduction step. It keeps track of each step, called a playout, and marks an additional variable referred to as a big playout. These occur after a certain number of playouts. This bare prover is improved on with Monte-Carlo guidance by implementing reward values for each of the nodes. It is then combined with policy learning, which interacts with the probability of switching nodes, and value learning, which assigns values to the big step based on whether or not a proof was found at the node. With the model trained off these aspects, the prover was tested with M2k, a subset of 2003 problems from the dataset of mathematical problems in Miz40. After 20 trials with learning, the improved prover was able to solve 1235 problems, compared to the bare prover solving 434 problems. The experiment results illustrate an increase in efficiency by testing each implementation individually, each trial resulting in more proofs solved.

Challenges in Implementing AI

On the other hand, when using machine learning as an AATP, it is difficult to understand the reasoning behind its decisions in outputting a proof. Referred to as the black box problem, it leaves other loose ends in the proving process, such as whether or not the machine properly followed the axioms or how it reasoned through the proof. This prevents mathematicians and computer scientists from following its reasoning to find where it went wrong. Furthermore, while its function is constructed to be human-like, its training method is very different from humans. It is forced to learn a large set of equations quickly, different from how children are taught math as they grow up. It is also notable that ANNs depend on a large dataset to learn from. It is often difficult to find a large dataset to teach AI in mathematics. Tao points out, mathematicians have only published their successes, frequently omitting the failures which led up to their findings. For example, when used as an AATP, the machine may stop partially through the proof because it was not taught what to do when the proof goes wrong. Then, ANNs lack this aspect and are left with half finished proofs.

As a result, the output is not consistently a well-formed formula, whereas the output of a rule-based prover always is. In its current state, ANNs only handle a subset of what current generation theorem provers can handle. While it has been shown that these tools are more

efficient, it is important to state that the problems in the experiment, such as the dataset Miz40, have already been proven by current state of the art ATPs. In future cases, the reliability of ANN must be solidified in order for it to be used as a tool in mathematical proving. The current model is also limited in that it depends on pre-existing data. Returning to Gödel's incompleteness theorem, we can also consider that there is a lingering danger in the foundations of ATP. It may run into issues, because of the existing statements which cannot be proved or disproved.

Benefits and Pathways Towards Change

To illustrate the possible implementations of theorem proving in AI, we will examine a problem similar to the Equational Theories project. This project conducted work in experimental mathematics to discover relationships between the laws of algebra, which was worked on by a large group of volunteers who contributed small portions of the solution. In the following strategy, machine learning will be implemented directly in developing proofs. Consider the following problem: given a dataset of problems to solve, what are the possible methods to prove as many problems as possible? Is it possible that the entire set could be solved, and how much resources would it take to solve? Given a result, how reliable is this method? As described in Tao's lecture, the approach to the Equational Theories project used a "top-down" strategy, breaking down a large problem into smaller ones. Following this strategy, one method to solve this issue is to approach the problem with a community of volunteers, who will use both ITP and machine learning AATP in order to create individual solutions to the problem. The dataset is divided into smaller parts, which can be worked on by a large group of people, each taking a smaller part to work on. This benefit is made possible by machines because each volunteer is working through their part by using some ATP to come up with solutions.

In this example, we consider using machine learning in the AATP step. First, the experimentalist will input a problem and its axioms into a machine learning AATP. Then, given the idea, it can be run through a rule-based theorem prover, whose reasoning is clear to see and the proof can be verified. The human aspect of this process solves the issue of identifying interesting proofs, while taking advantage of the resources provided by ANN in order to make the process collaborative and efficient. This method can prove more problems compared to a rule-based prover, focuses more energy into computing rather than the experimentalist, and integrates the strengths and weaknesses of machines and humans. The prover comes up with ideas and attempts them, while still administered and overseen by an experimentalist. Even incomplete outcomes may be analyzed by humans. However, this method has some limitations. It still relies on the correctness of the code made by each volunteer, requires that the ANN theorem prover outputs a well formed formula, and that what it outputs can be read by the experimentalist after verification. Additionally, it may have a large amount of outputs, which make it difficult to analyze its work.

We can also consider that it is possible to create a set of agents which complete subsets of problems, rather than having volunteers facilitate the process. In this case, ANN would still struggle to define an interesting proof. We can extend these examples outside the realm of mathematics, as companies seek to implement AI tools in the workplace. The concept of agents, collaboration with ITPs, and the ideas created by AATPs are opportunities to increase efficiency.

This method relies more on the future potential of ANN, as it remains a strong setback that it is difficult to identify interesting proofs. Even though its weakness of outputting inaccurate proofs may be verified by a rule-based theorem prover, both of these systems struggle to describe proofs. Given ANNs developed to solve this issue, then the most important aspect in using these tools is that their output can be understood by humans.

Conclusions

In this project, it has been demonstrated that ANNs provide a new approach to AATP and alternative methods in the process of ITP. In mathematics, the development of AI technology opens up new possibilities in experimental mathematics, allowing a community of members to work collaboratively on a project. The efficiency of machine learning in the role of ATP has been experimentally shown to be quicker compared to rule-based theorem provers. Because of the unique structure of ANNs, it may be possible to utilize these theorem provers to implement human-like thought into an algorithm. With this, theorem proving in AI still has limiting weaknesses. It relies on lots of information before it can be used, as highlighted in its training process. The results of the incompleteness theorems seem to avoid the major drawbacks, but the current state of this result poses challenges in completely formalizing mathematics. Furthermore, if there is no dataset to reference, it would be difficult to effectively use theorem proving in AI. In potential implementations, it is found that the applications where AI can be included in ATP allow for greater productivity in solving proofs and in tackling larger problems. In the future, the development of ANNs may better attack the problems in analyzing proofs, and in displaying their results to be comprehensible by humans.

Bibliography

[1] P. Raatikainen, “Gödel’s Incompleteness Theorems (Stanford Encyclopedia of Philosophy),” Stanford.edu, Nov. 11, 2013. https://plato.stanford.edu/entries/goedel-incompleteness/

[2] P. Radke, “Monte Carlo Tree Search: A Guide | Built In,” builtin.com, Jul. 20, 2023. https://builtin.com/machine-learning/monte-carlo-tree-search

[3] Amazon Web Services, “What is Reinforcement Learning? - Reinforcement Learning Explained - AWS,” Amazon Web Services, Inc. https://aws.amazon.com/what-is/reinforcement-learning/

[4] M. Pantsar, “Theorem proving in artificial neural networks: new frontiers in mathematical AI,” European Journal for Philosophy of Science, vol. 14, no. 1, Jan. 2024, doi: https://doi.org/10.1007/s13194-024-00569-6.

[5] Simons Foundation, “Terence Tao - Machine-Assisted Proofs (February 19, 2025),” YouTube, Feb. 20, 2025. https://www.youtube.com/watch?v=5ZIIGLiQWNM (accessed Mar. 29, 2025).

[6] Cezary Kaliszyk, J. Urban, Henryk Michalewski, and Miroslav Olšák, “Reinforcement Learning of Theorem Proving,” Neural Information Processing Systems, vol. 31, pp. 8822–8833, May 2018, doi: https://proceedings.neurips.cc/paper/2018/file/55acf8539596d25624059980986aaa78-Paper.pdf